

محمدرضا شریفی

۴۰۱۱۰۱۹۳۲

طراحی سیستم های میکروپروسسوری

فاز پنجم

مقدمه و پیاده سازی OpenMP

در فاز پنجم از پروژه، با هدف استخراج حداکثر توان پردازشی از سخت افزار، تکنیک پردازش Multi-threading با استفاده از OpenMP به کد اضافه شده است. یکی از الزامات اساسی این فاز، حفظ و تجمیع تمامی بهینه سازی های پیاده سازی شده در فاز های پیشین است؛ لذا در تابع نهایی (matrix_mult_omp_vec) تمامی تکنیک های پیشین اعم از استفاده از Pointers، تخصیص رجیستر ها، ترانهاد کردن ماتریس، بهینه سازی حافظه کش، باز کردن حلقه (Loop Unrolling) و همچنین استفاده از افزونه های برداری کامپایلر (GNU-C Vector Extensions / AVX) به دقت لحاظ شده اند.

برای موازی سازی این ساختار پیچیده، از دستور #pragma omp parallel for schedule(static) استفاده شده است. ابتدا عملیات ترانهاد کردن ماتریس به صورت موازی بین تردها تقسیم شده است. سپس در بخش ضرب اصلی، برای جلوگیری از overhead ایجاد و خاتمه ای مداوم تردها، ناحیه موازی تنها یک بار در خارج از حلقه های محاسباتی باز شده است. استفاده از زمان بندی ایستا (schedule(static)) تضمین می کند که بلوک های داده به صورت پیوسته و با بالاترین تطابق با حافظه کش (Cache Locality) به هر هسته اختصاص یابند. همچنین برای اندازه گیری دقیق زمان دنیای واقعی (Wall-Clock Time) به جای زمان پردازنده، تابع clock() با تابع دقیق تر omp_get_wtime() جایگزین گردیده است. جهت کامپایل صحیح این فاز و فعال سازی قابلیت های برداری و موازی سازی، از دستور زیر در ترمینال استفاده شده است:

```
gcc -O0 -mavx2 -Wall -Wextra -fopenmp -o phase5 main.c
```

اعداد نوشته شده در طول این گزارش یا میانگین نتایج خروجی هستند و یا نتایجی هستی که به آن عدد زمانی همگرا شده است.

کد نوشته شده:

```
346 void matrix_mult_omp_vec(int n, double* a, double* b, double* c) {
347     int i, j;
348     int BS = 64; // Block Size
349
350     // Allocate transpose buffer
351     double* b_t = (double*)_aligned_malloc(n * n * sizeof(double), 64);
352     if (b_t == NULL) return;
353
354     // 1. Parallelize the transpose
355     #pragma omp parallel for private(j) schedule(static)
356     for (i = 0; i < n; i++) {
357         double* p_b = b + i * n;
358         for (j = 0; j < n; j++) {
359             b_t[j * n + i] = p_b[j];
360         }
361     }
362
363     int vec_limit = n - (n % 16);
364
365     // 2. Parallelize the multiplication
366     // We open the parallel region ONCE to avoid thread fork/join overhead on every block
367     #pragma omp parallel
368     {
369         // Thread-private variables
370         register int jj, j_inner, k;
371         register double* at, *bt;
372         register double cij;
373
374         // Keep the cache-friendly block loop strictly on the OUTSIDE
375         for (jj = 0; jj < n; jj += BS) {
376
377             // Distribute the rows of 'A' among the threads for the current block.
378             // Static scheduling ensures maximum cache contiguity and zero OpenMP overhead.
379             #pragma omp for schedule(static)
380             for (i = 0; i < n; i++) {
381
382                 for (j_inner = jj; j_inner < jj + BS && j_inner < n; j_inner++) {
383                     at = a + i * n;
384                     bt = b_t + j_inner * n;
385
386                     v4df sum0 = {0.0, 0.0, 0.0, 0.0};
387                     v4df sum1 = {0.0, 0.0, 0.0, 0.0};
388                     v4df sum2 = {0.0, 0.0, 0.0, 0.0};
389                     v4df sum3 = {0.0, 0.0, 0.0, 0.0};
390
391                     register v4df* vec_at = (v4df*)at;
392                     register v4df* vec_bt = (v4df*)bt;
393
394                     for (k = 0; k < vec_limit; k += 16) {
395                         sum0 += vec_at[0] * vec_bt[0];
396                         sum1 += vec_at[1] * vec_bt[1];
397                         sum2 += vec_at[2] * vec_bt[2];
398                         sum3 += vec_at[3] * vec_bt[3];
399
400                         vec_at += 4;
401                         vec_bt += 4;
402                     }
403
404                     v4df final_vec = sum0 + sum1 + sum2 + sum3;
405                     cij = final_vec[0] + final_vec[1] + final_vec[2] + final_vec[3];
406
407                     // Tail loop for leftovers
408                     at = (double*)vec_at;
409                     bt = (double*)vec_bt;
410                     for (; k < n; k++, at++, bt++) {
411                         cij += (*at) * (*bt);
412                     }
413
414                     c[i * n + j_inner] = cij;
415                 }
416             }
417         } // End of parallel region
418
419         _aligned_free(b_t);
420     }
421 }
422 // =====
```

تحلیل عملکرد: نسخه single-thread در برابر multi-thread

در جدول زیر، عملکرد بهترین کد فاز چهارم با کد فاز پنجم مقایسه شده است.

n	matrix_mult_cache_friendly_vec (single-thread)	Matrix_mult_omp_vec (multi-thread)	Gain
1024	0.28	0.08	3.5
2048	2.25	0.68	3.30
4096	21.10	6.33	3.34

نتیجه به دست آمده:

نتایج حاصل از اجرای موازی به وضوح نشان‌دهنده استفاده موثر از چندین هسته پردازشی است. همان‌طور که در مباحث کلاس نیز مطرح شد، با وجود استفاده از پردازنده چهارهسته‌ای انتظار افزایش سرعت دقیقاً خطی (گین دقیقاً برابر با ۴) دور از واقعیت است. این امر به دلیل وجود محدودیت‌هایی نظیر سرشار ناشی از ایجاد و مدیریت تردها توسط سیستم‌عامل، مسائل مربوط به همگام‌سازی، و مهم‌تر از همه، اشباع شدن پهنای باند حافظه اصلی در ابعاد بزرگ است. با این حال، دستیابی به گین در حدود ۳.۴ برابر، کاملاً طبیعی، قابل قبول و نشان‌دهنده‌ی یک پیاده‌سازی موفق از معماری Multi-threading می‌باشد.

گزارش خروجی‌های کنسول:

N=1024:

```
Do you want to run matrix_mult_omp_vec (y/n)? y
Execution time of matrix_mult_omp_vec = 0.09 s
End Of Execution
```

N=2048:

```
Do you want to run matrix_mult_omp_vec (y/n)? y
Execution time of matrix_mult_omp_vec = 0.69 s
End Of Execution
```

N=4096:

```
Do you want to run matrix_mult_omp_vec (y/n)? y
Execution time of matrix_mult_omp_vec = 6.31 s
End Of Execution
```

تحلیل نهایی: مقایسه فاز صفر با فاز پنجم

برای درک بهتر ارزش تکنیک‌های به کار رفته در طول این ترم، در جدول زیر کد پایه و اولیه پروژه که فاقد هرگونه بهینه‌سازی بود، با کد نهایی فاز پنجم مقایسه شده است.

n	Matrix_mult_omp_vec (Phase 5 - Final)	Matrix_mult_index (Phase 0 - Reference)	Total Gain
2048	0.68	77.8	114.4

N=2048:

```
Do you want to run matrix_mult_index (y/n)? y  
  
Execution time of matrix_mult_index = 82.49 s
```

نتیجه‌گیری نهایی:

همان‌طور که از جدول فوق مشخص است، تجمیع روش‌های مهندسی نرم‌افزار در سطح سخت‌افزار توانسته است زمان اجرا را به طرز چشم‌گیری کاهش دهد. الگوریتم پایه به دلیل دسترسی‌های تصادفی به حافظه، عدم استفاده از کش، عدم بهره‌گیری از رجیسترهای برداری SIMD و متمرکز بودن روی یک هسته، در ابعاد بزرگ (مانند ۲۰۴۸) کاملاً ناکارآمد است. اما با پیمودن فازهای مختلف و اعمال تکنیک‌های Cache-Friendly، Vectorization و در نهایت Multi-threading، موفق شدیم کدی تولید کنیم که از تمامی پتانسیل سلسله‌مراتب حافظه و هسته‌های فیزیکی پردازنده استفاده کرده و گین نهایی فوق‌العاده‌ای در حدود ۱۱۵ برابر را به ثبت برساند. البته که لازم به ذکر است گین کلی متغیر و به اندازه ماتریکس نیز وابسته است.

اگر حاصل ضرب گین حدودی هر فاز را بخواهیم به عنوان گین نهایی گزارش کنیم به عدد ۳۵۷ میرسیم:

$$\begin{aligned} Total\ Gain &= Gain_{phase1} * Gain_{phase2} * Gain_{phase3} * Gain_{phase4} * Gain_{phase5} \\ &\rightarrow 3 * 5 * 2 * 3.5 * 3.4 = 357 \end{aligned}$$